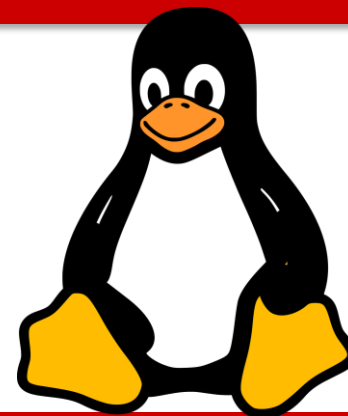


Introduction to Linux

for Bioinformatics & HPCC



Mahmoud A. Noamaan, PhD

Teaching & Research Assistant

Department of Mathematics, Faculty of Science

Cairo University



```
$ ssh you@hpc.cu.edu.eg  
$ module load bioinfo  
$ sbatch align.slurm  
$ squeue -u $USER
```

Computational Chemistry • Molecular Modeling • Numerical Analysis • Applied Mathematics • Scientific Computing

Lecture Agenda

Part 1: Introduction to basic topics in Linux

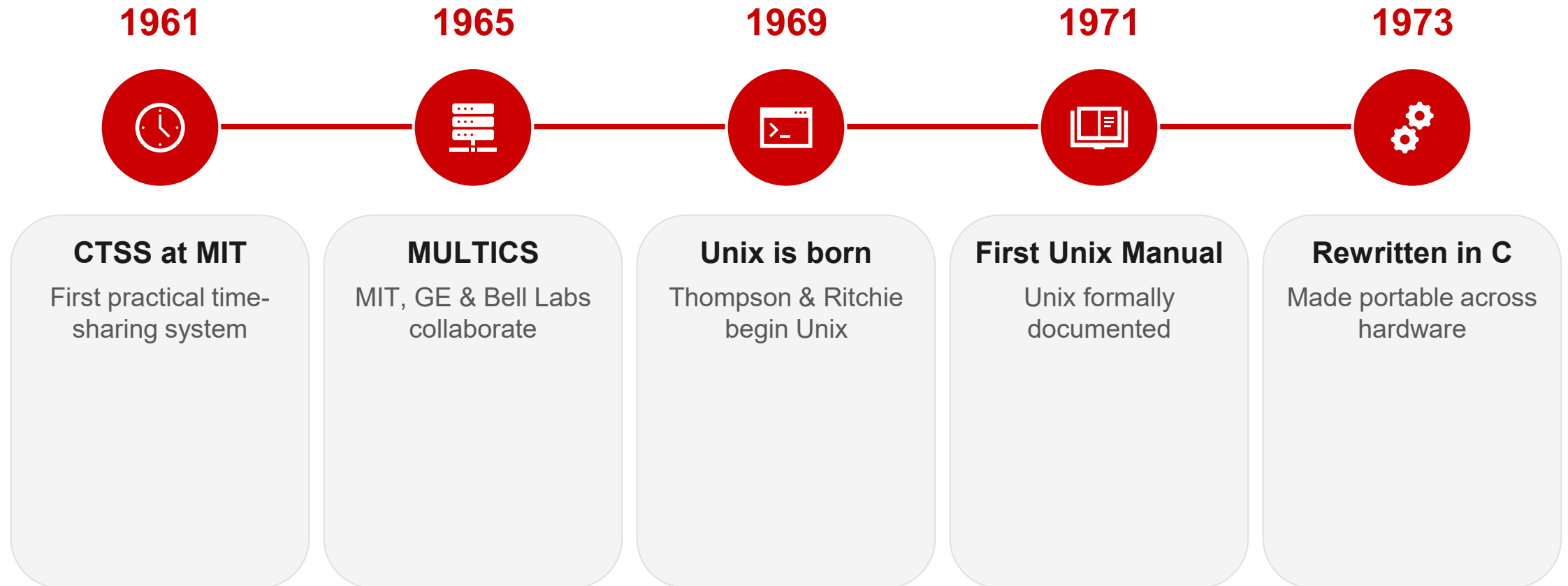
- ❖ About Linux
- ❖ Working with Files and Directories
- ❖ Linux Essential Commands (Part 1)

Part 2: Introduction to more advanced topics in Linux

- ❖ Linux Essential Commands (Part 2)
- ❖ Text Editors in Linux
- ❖ Linux Environment Variables
- ❖ Basic Bash Scripting in Linux

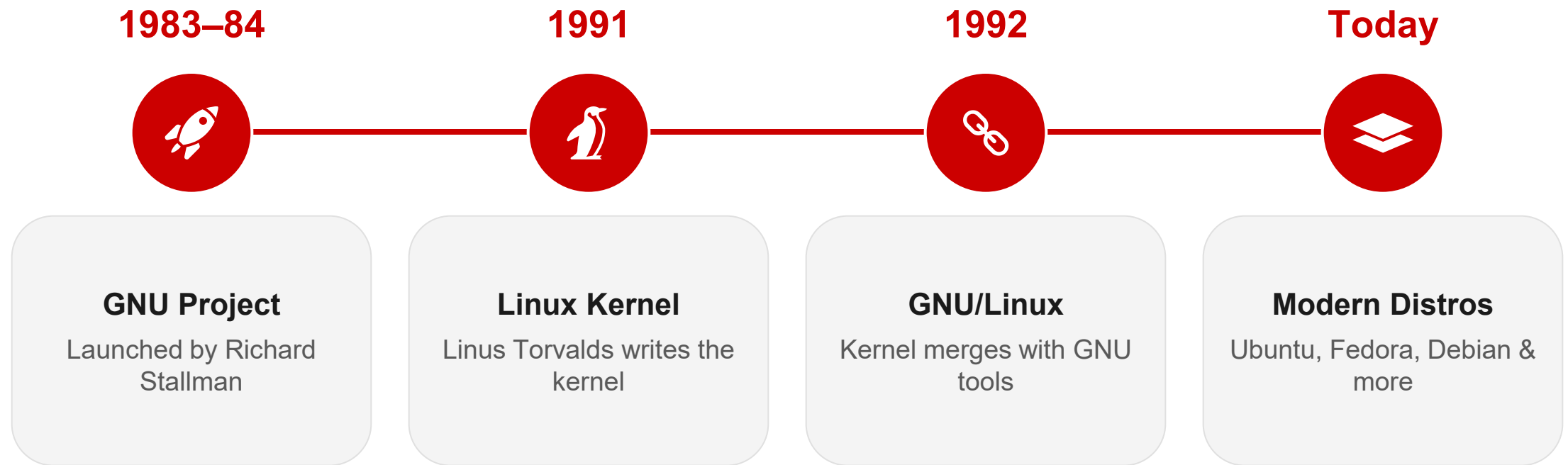
A brief history of Linux

The Unix Era 1961 – 1973



A brief history of Linux

The Birth of Linux 1983 – Today



Why it matters: Every modern Linux system unites Torvalds' kernel with GNU tools — powering most of the world's servers, Android phones, and the cloud.

A brief history of Linux

GNU Project:

- Richard Stallman started the GNU project in 1983
 - After AT&T imposed commercial licensing on Unix Operating System
 - Reprogrammed some of the Unix Tools. running on a small kernel (TRIX)
 - Needed more advanced and reliable OS Kernel for his programs

GNU tool and libraries
+
Linux Kernel
=
GNU/Linux

Linux:

- Linus Torvalds created the Linux kernel in 1991
 - Linus first released Linux under his own license, then relicensed the kernel under the GPLv2 in 1992
 - *Requires that the source code remains freely available to everyone*

Linux Distributions

Linux has various distributions (Distros):

- Why many distributions?

- Server vs. Desktop
- Commercial user support
- Special hardware support

- # Debian/RedHat/SUSE/...

- The major difference is the software package management on these distros.

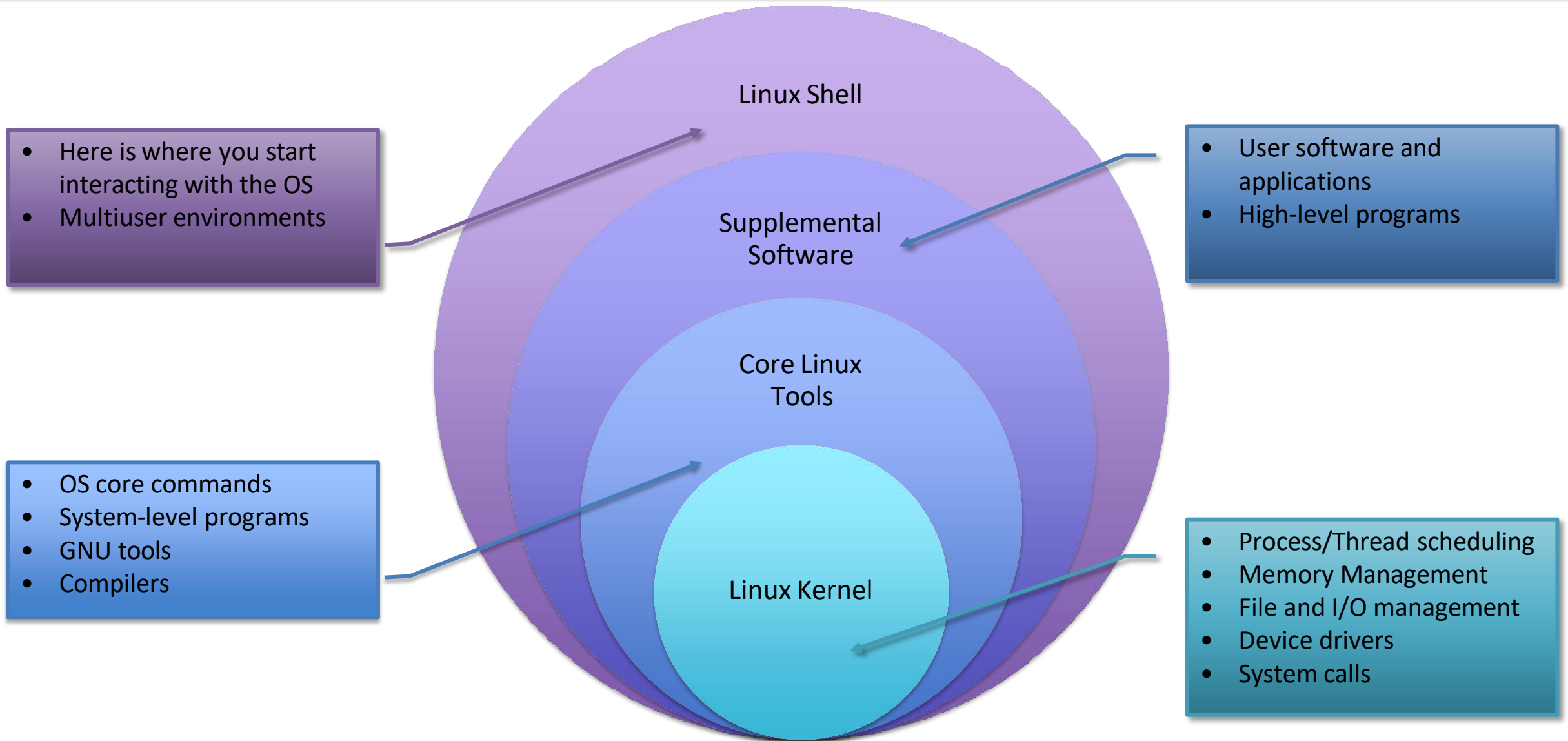


Linux distribution family

sorted by base distro with derivatives



Structure of Linux OS



Linux Shell

What is the shell?

- The shell is a program that takes commands from user's keyboard and passes them to the operating system to execute.
 - Many shell programs are available for Linux:
 - `bash`, `sh`, `csh`, `tcsh`, `ksh`, `zsh`, ...
 - For this tutorial, we will use `bash`, but other shells are conceptually similar

```
quanah:$ command
```

What is Terminal?

- Terminal is a program that opens in a window and lets users interact with the shell.

Working with Files and Directories

Understanding Linux Directory Structure



Basic Directory Operations

Where am I?

- **pwd** command to print working directory

```
quanah:$ pwd  
/home/username
```

- **ls** command to list contents of the current directory

```
quanah:$ ls  
test1.txt  
Mydir1  
mydir2
```

Basic Directory Operations

Make/Remove/Go into a directory?

- **mkdir** command to make a new

```
quanah:$ mkdir my_new_dir
quanah:$ ls
my_new_dir
```

- **cd** command to change into a directory

```
quanah:$ cd my_new_dir
```

- **rmdir** command to remove an empty directory

```
quanah:$ rmdir my_new_dir
```

Basic Directory Operations

More about `ls`. (Commands with options)

- `ls -a` (`--all`) list all files including hidden files

```
quanah:$ ls -a
```

- `ls -l` show file details

```
quanah:$ ls -l
```

- Combine multiple options for a command

```
quanah:$ ls -a -l
```

```
quanah:$ ls -al
```

Special Directory Names (Characters)

- Return to home directory

```
quanah:$ cd  
quanah:$ cd ~  
quanah:$ cd ~/
```

- Current directory (.)

```
quanah:$ ls .  
quanah:$ ls ./
```

- Parent directory

```
quanah:$ cd ..  
quanah:$ cd ../  
quanah:$ ls ../..  
quanah:$ cd ../../mydir/
```

Absolute VS. Relative path

Absolute Path of a file/directory:

- Always starts with / and goes all the way to the file or directory

```
quanah:$ cat /home/Noamaan/mydir/file1.txt
```

Relative Path of a file/directory:

- It defines from the current or parent directory
 - For instance:

```
quanah:$ cat ../mydir/file1.txt
```

File and Directory Names

Naming: What to use:

- Letters and numbers [A-Z a-z 0-9]:
 - File and directory names are case sensitive (e.g. 'M' vs 'm')
- (.) can be used anywhere in the file/directory name
 - Using (.) at the beginning of the file/directory name makes it hidden
 - Using (.) alone will be considered as current working directory

- A-Z
- A-z
- 0-9
- dot (.)
- dash (-)
- underscore (_)

What to Avoid:

- Avoid spaces in the file names
 - "my file name.txt" → "my_file_name.txt"
- Avoid these characters:
 - () " ' ? \$ * \ / :

- () parenthesis
- " double quotes
- ' single quote
- ? Question mark
- \$ dollar sign
- * asterisk
- \ back slash
- / slash
- : colon

A few Basic File Operations

Let's learn a few file operations before Exercise #1

- **touch** creates an empty file (Not the primary job!)

```
quanah:$ touch my_file  
quanah:$ ls  
my_file
```

- **echo** prints its arguments to the standard output
- **>** redirects the standard output to a file (Creates/Rewrite the file)
- **>>** redirects the standard output to a file (Append to the file)

```
quanah:$ echo "hello world!" > hello.txt  
quanah:$ echo "second line" >> hello.txt
```

A few Basic File Operations (Cont.)

Let's learn some file operations before Exercise #1

- **cat** shows the content of a file(s)

```
quanah:$ cat hello.txt  
Hello world!  
second line
```

- **wc** counts number of lines, words, and characters in a text

```
file  
quanah:$ wc hello.txt  
2  4 25 hello.txt  
quanah:$ wc -l hello.txt  
2 hello.txt  
quanah:$ wc -w hello.txt  
4 hello.txt
```

File and Directory Operations

Copy Files and Directories:

- **cp** command to make a copy of a file(s) or directory(s)
 - **-r**: Recursively copy the directories and subdirectories as well

```
quanah:$ ls
file1.txt
quanah:$ cp file1.txt file2.txt
quanah:$ ls
file1.txt  file2.txt
quanah:$ cp -r ../mydir ./
quanah:$ cp ../mydir2/* /home/bob/
quanah:$ cp *.*.txt ~/text_dir/
```

File and Directory Operations

Move/Rename Files and Directories:

- **mv** command moves file(s)/directory(s) to a destination. On the same filesystem it just renames (relinks the inode) — no data is copied; only across filesystems does it copy then delete.

```
quanah:$ ls
file1.txt
quanah:$ mv file1.txt file2.txt
quanah:$ ls
file2.txt
quanah:$ mv mydir/ ../temp/
quanah:$ mv ../docs/*.pdf ./
```

File and Directory Operations

Deleting Files and Directories:

- **rm** command to remove file(s) and directory(s) permanently
 - Removing files and directories by this command may not be recovered easily (This is an exception for `/home` directory on HPCC clusters)
 - **-r**: Recursively remove directories and their contents

```
quanah:$ ls
file1.txt  file2.txt
quanah:$ rm file1.txt
quanah:$ ls
file2.txt
quanah:$ rm ../mydir/*
quanah:$ rm -r ../mydir/
```

Output Redirection

- So far, all the commands that we used sent their output to the screen. We can control this in Linux:
 - < redirects to the standard input
 - > redirects the standard output
 - >> appends the standard output
 - 1> redirects the standard output (equals to >)
 - 2> redirects the standard error
 - &> redirects the standard output and standard error
 - | (pipe) pass the output of one command to another

Exercise #2

1. Return to your home directory
2. Create a new directory and name it “exercise2”
3. Go to the “exercise2” directory
4. Copy the “test1.txt” file from the “exercise1” directory to the “exercise2” directory
5. Make a new directory under “exercise2” and name it “subdir2”
6. Rename the file “test1.txt” to “test2.txt”
7. List the contents of “exercise2” and redirect the output to “ls.out” file
8. Remove everything under the “exercise2” directory

Linux Essential Commands 2

File/Directory Ownership and Permissions:

- Every file/directory belongs to a specific user or a group of users
- Every user/group may have permissions to `read`, `write`, and/or `execute`

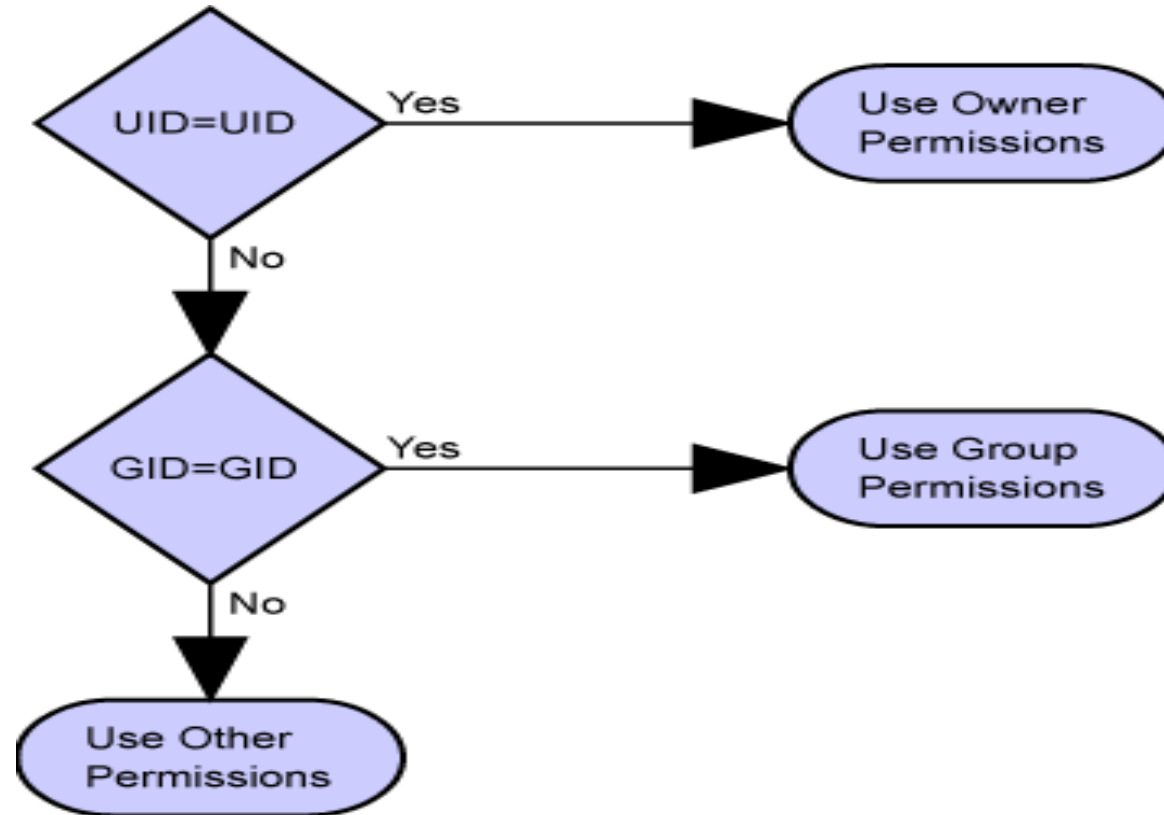
User	Group	Other
r w x	r w x	r w x

- If you set **write** permission for a directory, you can (create/delete) new entries
- If you set **read** permission for a directory, you can list (`ls`) the contents
- If you set **execute** permission for a directory, you can (`cd`) into the directory

Permission Notations

Permission		Access for a File	Access for a Directory
Read		You can display file contents and copy the file.	You can list the directory contents with the ls command
Write		You can modify the file contents.	If you also have execute access, you can add and delete files in the directory.
Execute		You can execute the file if it is an executable. You can execute a shell script if you also have read and execute permissions.	You can use the cd command to access the directory. If you also have read access, you can run the ls -l command on the directory to list contents.

Determining Permissions



Changing the Permissions

```
chmod permission filename
```

- Permissions are specified in either
 - Symbolic mode
 - Who
 - u: Owner permissions
 - g: Group permissions
 - o: Other permissions
 - a: all permissions

Changing the Permissions

- Permissions are specified in either
 - Symbolic mode
 - Operator
 - + Add permissions
 - - Remove permissions
 - = Assign permissions absolutely
 - Permissions
 - r: read
 - w: write
 - x: execute
- Permissions are specified in either
 - Octal mode
 - 4 read
 - 2 write
 - 1 execute

Linux Essential Commands 2

File/Directory Ownership and Permission Examples:

- **chmod** command changes the `rwX` mode bits of a file or directory
 - `+/−`: adds or removes the mode bits
 - **u**: Sets the permissions for the owner of the file/directory
 - **g**: Sets the permissions for the group that the owner belongs to
 - **o**: Sets the permissions for the other users
 - **a**: Sets the same permissions for user/group/other

```
quanah:$ chmod +x script.sh
quanah:$ chmod g+rx my_program
quanah:$ chmod a-r my_docs
quanah:$ chmod 755 ./mydir
```

Linux Essential Commands 2

File/Directory Ownership and Permission:

- With **stat** or **ls -l** commands you can check the ownership and permissions of a file or directory
- **whoami** command Displays the username of the current user
- **groups** command prints the groups a user belongs to

Linux Essential Commands (Part 1)

Linux Essential Commands 1

Check the manual page of a command:

- **man** command to search and open the manual page of a Linux command

```
quanah:$ man ls  
quanah:$ man cp
```

- **/word** search the man page for the text 'word'
- **n** search for the next found match
- **N** search for the previous found match
- **g** Go to the beginning of the manual page
- **G** Go to the end of the manual page
- **q** Exit (quit)

Linux Essential Commands 1

Other useful commands to find out more:

- Several built-in references are available within the operating system.
 - **info** more detailed tutorials and examples.

```
quanah:$ info cp  
quanah:$ apropos mkdir
```

- Many reference guides are available.
 - Search the web for “reference” – for example “bash reference”.
 - Specifically for bash, try finding the “bash advanced scripting guide” (but only when you are ready for the advanced stuff!
 - Guides are available for other shells and commands also.

Linux Essential Commands 1

Paging through long text files/outputs:

- **more** command for paging through text one screenful at a time

```
quanah:$ more ./README
quanah:$ cat ./README | more
```

- **Less** command is an improved version of **more** command which allows backward movement in the file as well as forward movement

```
quanah:$ less ./README
quanah:$ cat ./README | less
```

Linux Essential Commands 1

Display a partial content of a file/output:

- **head** command to see first lines of a file (10 lines by default)
 - **-n <NUM>**: First NUM number of a file/output

```
quanah:$ head -n 15 ./README
quanah:$ cat ./README | head -3
```

- **tail** command to see last lines of a file (10 lines by default)
 - **-n <NUM>**: Last NUM number of a file/output

```
quanah:$ tail -n 20 ./README
quanah:$ cat ./README | tail -5
```

Linux Essential Commands 1

Searching for pattern(s) in files/outputs:

- **grep** command to print lines in a file matching a pattern
 - **-i**: Ignore case distinctions in both the PATTERN and the input files
 - **-v**: Invert the sense of matching, to select non-matching lines

```
quanah:$ grep -c ">" seqs.fasta    # count sequences in a FASTA
1428
quanah:$ grep -c "^@" reads.fastq  # count reads? risky!
# quality lines can also start with @ -- safer to use:
quanah:$ echo $(( $(wc -l < reads.fastq) / 4 ))
50000
```

Linux Essential Commands 1

History of your commands:

- **history** command to see the commands you have typed so far.
 - Your command are saved to the **.bash_history** file under your home directory.

```
quanah:$ history | less
quanah:$ history | tail -15
quanah:$ history | grep "cp"
```

Part 2: Introduction to more advanced topics in Linux

- ❖ Linux Essential Commands (Part 2)
- ❖ Text Editors in Linux
- ❖ Linux Environment Variables
- ❖ Basic Bash Scripting in Linux

Review

Command	Description
pwd	Prints Current Working Directory
ls	Lists the contents of a directory
cd	Change the current path to the destination directory
mkdir	Makes a new directory
rmdir	Removes an empty directory
cp	Copy file or directory
mv	Move/Rename file or directory
rm	Delete file or directory
cat	Concatenates and prints the contents a file

Review

Command	Description
echo	Write arguments to the standard output
wc	word, line, character, and byte count
man	Search and open the manual page of a Linux command
more	Paging through text one screenful at a time
less	Improved version of more allows backward/forward movement
head	Display first lines of a file
tail	Display last lines of a file
grep	Print lines in a file matching a pattern
history	See the commands you have typed so far

Linux Essential Commands (Part 2)

Linux Essential Commands 2

Define the file type:

- Unlike Windows, files extensions rarely define the type of a file in Linux.
 - For example: “file.txt” may not be a TEXT file.
- **file** command displays the file type along with a brief summary of the file contents.

```
quanah:$ file myfile
myfile: ASCII text
quanah:$ file /home/Noamaan/program.py
/home/Noamaan/program.py: Python script, ASCII text executable
quanah:$ file test.tar.gz
test.tar.gz : gzip compressed data, from Unix, last modified: Wed
Sep  4 14:04:10 2019
```

Linux Essential Commands 2

Compression and File Packaging:

- **zip** command packages and compresses files and directories
 - **zip** [OPTIONS] zip_file file_dir1 file_dir2 ...
 - **-r** : Add the directories and subdirectories contents into the zip file

```
quanah:$ ls
mydir test1.txt
quanah:$ zip -r archive.zip ./*
adding: ./test1.txt (deflated 62%)
adding: ./mydir/ (deflated 0%)
quanah:$ ls
archive.zip mydir test1.txt
```

Linux Essential Commands 2

Compression and File Packaging:

- **unzip** command lists and extracts the contents of a zipped file
 - **-l** : Lists the contents of a zipped file

```
quanah:$ ls
archive.zip
quanah:$ unzip archive.zip
quanah:$ ls
archive.zip mydir test1.txt
```

Linux Essential Commands 2

Compression and File Packaging:

- Other compression commands available in Linux

Command	Description	Decompression	File Ext.
zip	Packages and compresses files and directories	unzip	.zip
gzip	A GNU tool for compressing or expanding files/directories	gunzip gzip -d	.gz
bzip2	Compresses files using the Burrows-Wheeler block sorting text compression algorithm.	bunzip2 bzip2 -d	.bz, .bz2
xz	Similar to gzip and bzip2	unxz	.xz

Linux Essential Commands 2

Archiving:

- **tar** command saves many files and directories into a single “archive” file
 - **tar** OPTIONS dest_file src1 src2 ...
 - **-f** define the archive file path/name
 - **-c** Create a new archive
 - **-r** Append file(s) to an existing archive
 - **-x** Extract the contents of an archive file
 - **-z** Compress archive file with gzip
 - **-j** Compress archive file with bzip2
 - **-v** verbosely list files processed

Linux Essential Commands 2

Archiving:

- **tar** command

examples:

```
quanah:$ ls
mydir test1.txt
quanah:$ tar -cf myarchive.tar ./*
quanah:$ ls
myarchive.tar mydir test1.txt
quanah:$ tar -xvf myarchive.tar
test1.txt
mydir/
quanah:$ tar -czf docs.tar.gz /home/Noamaan/docs
quanah:$ ls
Docs.tar.gz myarchive.tar mydir test1.txt
quanah:$ tar -xzf docs.tar.gz
```

Linux Essential Commands 2

Download files from internet:

- **wget** command downloads files from internet
 - **-O** : (capital O) defined the name of the destination file on your system

```
quanah:$ wget "https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh" -O miniconda3.sh
```

Activate / load your tools each session:

```
quanah:$ conda activate bio-env # or: mamba activate bio-env
(bio-env) quanah:$ samtools --version # prompt shows the active env
quanah:$ module load bwa/0.7.17 # HPC: use preinstalled software
```


Software Modules on HPC

Environment modules make different software packages and versions available — without installing them in your home directory.

Common module commands

```
quanah:$ module avail           # list available modules
quanah:$ module spider bwa      # search for software
quanah:$ module load bwa/0.7.17 # load software
quanah:$ module list            # check loaded modules
quanah:$ module unload bwa      # remove a module
quanah:$ module purge           # reset the environment
```

Load bio-tools by name

```
quanah:$ module load samtool
quanah:$ module load blast
quanah:$ module load gromacs
```

Heads-up

Module names and versions depend on the HPC system. Run `module avail` to see what your cluster provides.

Exercise #4

1. Go to your home directory
2. Create a new directory and name it “exercise4”
3. Go to the “exercise4” directory
4. Choose a small directory from your home directory
5. Try to archive and compress the directory by tar and save it under the “exercise4” directory.
6. Now try to decompress the tar file that you just created
7. Check the type of the file
8. Now try to untar the file

Text Editors in Linux

Text Editors in Linux

How to edit text files in Linux?

- There are many text editors available on Linux
 - **nano** is a small, simple and friendly editor
 - **vi/vim** is a powerful text editor which can be used to edit all kinds of text
 - **emacs** is part of the GNU project written by Richard Stallman
- In this training course we will cover **nano** and **vi/vim**
- Let's look into **nano** (*Demo*)

Text Editors in Linux

How does `vi/vim` work?

- **`vi/vim`** is a very popular text editor among programmers and system administrators
- It supports many programming and scripting languages
- Suitable for more advanced file editing
- **`vi/vim`** has two modes:
 1. **Insert mode:** which can be enabled by typing **`i`** (insert) or **`a`** (append)
 2. **Normal mode:** which will be enabled by pressing the **Esc** key on keyboard — the default mode for navigation & commands (**`:w`**, **`:q`**).

Text Editors in Linux

Some useful `vi/vim` commands:

command	description	command	description
!	Forces the action	i	insert
:q	quit	a	append
:q!	Force quit	x	Delete a character
:w	write	y[count]y	Yank (copy) [count] lines
:wq	Write and quit	d[count]d	Cut (Delete) [count] lines
:x	Write and quit	p	Paste after the current line

- Let's look into **`vim`** (*Demo*)

Exercise #5

1. Go to your home directory
2. Create a new directory and name it “exercise5”
3. Go to the “exercise5” directory
4. Open a new text file with the editor of your choice and type the lines below:

```
Exercise #5  
This is a Linux text editor exercise  
We finally made it!
```

5. Save the file and exit
6. Try to display the contents of the file on your screen
7. Change the permission of the file as below:
 - Set `read/write` permission for the owner
 - Set `read-only` permission to your group
 - Give a `read-only` permission to user `Noamaan` and then remove it.

Introduction to apt-get

- Ubuntu's package management system is derived from the same system used by the Debian GNU/Linux distribution.
- The package files contain all of the necessary files, meta-data, and instructions to implement a particular functionality or software application on your Ubuntu computer.
- The software management tools in Ubuntu will check dependencies automatically.

APT-Get

- The apt-get command is a powerful command-line tool used to work with Ubuntu's Advanced Packaging Tool (APT) performing such functions as
 - Installation of new software packages
 - Upgrade of existing software packages
 - Updating of the package list index
 - And even upgrading the entire Ubuntu system.

APT-Get Cont'd

- Install a Package

```
sudo apt-get install ksh
```

- Remove a Package

```
sudo apt-get remove ksh
```

```
sudo apt-get purge nmap
```

- * You may specify multiple packages to be installed or removed, separated by spaces.

APT-Get Cont'd

- Update the Package Index
 - Update is used to resynchronize the package index files from their sources.
 - The indexes of available packages are fetched from the specifies location(s) in `/etc/apt/sources.list`.
 - An update should always be performed before upgrade.

```
sudo apt-get update
```

APT-Get Cont'd

- Upgrade package
 - Install the newest versions of all packages currently installed on the system from the sources found in /etc/apt/sources.list.
 - An update must be performed first so that apt-get knows the new versions of packages available.

```
sudo apt-get upgrade
```

Basic Bash Scripting In Linux

Basic Bash Scripting in Linux

What is Bash Script?

- **Bash** script is an executable file contains Bash shell commands which could be used to automate and simplify things.
 - **Shell** script is a text file starts with (`# !`) followed by the path to the shell interpreter (i.e. `/bin/bash`)

```
quanah:$ vim myscript.sh
#!/bin/bash
echo "Hello World!"

quanah:$ chmod +x myscript.sh
quanah:$ ./myscript.sh
Hello World!
```

Basic Bash Scripting in Linux

Control flows

- The syntax of the `if-then-else` clause is as following:

- *Note: spaces are required inside `[]` — write `[ $var -eq 1 ]`, not `[$var -eq 1]`.*

```
if [ $var -eq 1 ]; then
```

```
...
```

```
elif [ $var -ne 1 ]; then
```

```
...
```

```
elif [ $var -gt 1 ]; then
```

```
...
```

```
elif [ $var -lt 1 ]; then
```

```
fi ...
```

Basic Bash Scripting in Linux

Loops

- The syntax of the `for...in` loop is as following:

```
for VARIABLE in 1 2 3 ... N; do
    ...
done
```

```
for i in $(seq 1 20); do tail -n 1 ./I${i}/initcond.log; done
```

```
for VARIABLE in file1 file2 file3; do
    ...
done
```

```
for VARIABLE in $(Linux command); do
    ...
done
```


Exercise #6

1. Go to your home directory
2. Create a new directory and name it “exercise6”
3. Go to the “exercise6” directory
4. Create a script file and name it “show_dirs.sh”
5. Program the “show_dirs.sh” to go over all the directories under your home directory and print the following message for each directory:
`/home/username contains --> the_directory_name`

then print the following line for “exercise6” directory
`This is the last directory --> excercise6`
6. Make your script file executable and run it.

Connecting to an HPC Cluster

What is SSH?

- SSH provides a secure command-line connection to a remote Linux computer.
- Most HPC systems are accessed through an SSH login node.
- To connect, you normally need:
 - The HPC hostname (e.g. hpc.university.edu)
 - Your username
 - A password or SSH key

Connect to the login node:

```
$ ssh username@hpc.university.edu
```

Using a non-default port:

```
$ ssh -p 2222 username@hpc.university.edu
```

Disconnect:

```
$ exit
```

How the connection works:



After logging in, your commands run on the remote HPC system — **not on your laptop.**

Login Node vs Compute Node

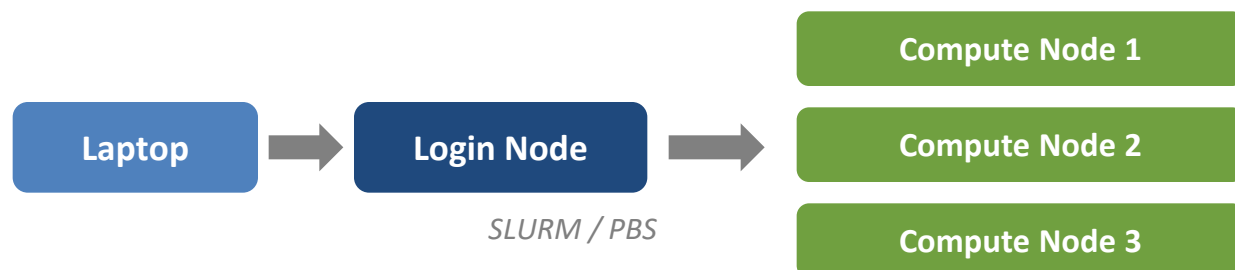
Login node	Compute node
Used for SSH access	Used for intensive calculations
Inspect and organize files	Run alignment, simulation, assembly
Edit scripts	Uses requested CPUs, memory & GPUs
Submit and monitor jobs	Allocated by the scheduler
Short, lightweight commands only	Runs scheduled workloads

Appropriate on a login node:

```
$ pwd
$ ls
$ head reads.fastq
$ vim analysis.sh
$ sbatch analysis.slurm
$ squeue -u $USER
```

Not appropriate on a login node:

```
$ bwa mem reference.fa reads.fastq
$ gromacs mdrun -deffnm md
$ python large_analysis.py
```



Do not run CPU-intensive, memory-intensive, GPU, or long-running analyses directly on the login node.

PBS vs SLURM on HPC Clusters

- ❖ **PBS** and **SLURM** are workload managers used to submit, schedule, and monitor jobs on HPC clusters.
- ❖ They perform the same main function but use different commands and job-script syntax.

PBS vs SLURM — Command Cheat-Sheet

Feature	PBS / OpenPBS / PBS Pro	SLURM
Full name	Portable Batch System	Simple Linux Utility for Resource Management
Job submission	qsub job.pbs	sbatch job.slurm
Check jobs	qstat	squeue
Cancel job	qdel JOB_ID	scancel JOB_ID
Job information	qstat -f JOB_ID	scontrol show job JOB_ID
Interactive allocation	qsub -l	salloc or srun --pty bash
Script directives	#PBS	#SBATCH
Queue terminology	Queue	Partition
Parallel launcher	Often mpirun or mpiexec	Usually srun
Job arrays	-t 1-10 (Torque/OpenPBS) / -J 1-10 (PBS Pro)	--array=1-10
Common usage	Academic and commercial HPC	Very common in modern large supercomputers

```

1 #!/bin/bash -l
2
3 #SBATCH --job-name=A296S ..... # Job name
4 #SBATCH --partition=cpu ..... # Partition
5 #SBATCH --nodes=2 ..... # Number of nodes
6 #SBATCH --ntasks-per-node=40 ..... # Number of cores
7 #SBATCH --output=job_%j.out ..... # Stdout (%j=jobId)
8 #SBATCH --error=error_%j.err ..... # Stderr (%j=jobId)
9 #SBATCH --time=24:00:00 ..... # Walltime
10 #SBATCH -A ..... # Account project
11
12
13 # Load necessary modules
14 #module load OpenMPI/4.1.4-GCC-11.3.0
15 module load GROMACS/2023.3-foss-2023a
16
17 init=step3_input
18 mini_prefix=step4.0_minimizationz
19 equi_prefix=step4.1_equilibration
20 equi_prefix_2=step4.2_equilibration
21 prod_prefix=step5_production
22
23 # for CONTINUATION
24 prev_step=step5_production
25 prod_step=step6_production
26

```

```

1 #!/bin/bash
2 #SBATCH -N gmx-2022.4
3 #PBS -l select=1:ncpus=24:mpiprocs=8:ompthreads=3
4 #PBS -q thin
5
6 cd $PBS_O_WORKDIR
7 module load /app/utis/modules/gromacs-2022.4
8
9 start_time=`date`
10 start_time_s=`date +%s`
11
12 mpirun gmx_mpi grompp -f step4.0_minimization.mdp -o step
13 mpirun gmx_mpi mdrun -v -deffnm step4.0_minimization -s:
14 mpirun gmx_mpi grompp -f step4.1_equilibration.mdp -o ste
15 mpirun gmx_mpi mdrun -v -deffnm step4.1_equilibration
16 mpirun gmx_mpi grompp -f step5_production.mdp -o step5_p
17 mpirun gmx_mpi mdrun -v -deffnm step5_production
18
19 echo Job Started: `date +%s` - $start_time_s
20 echo Job Finished: `date +%s` - $end_time_s
21 let end_time_s=`date +%s` - $start_time_s

```

THANK YOU! THIS IS THE END OF THE PRESENTATION.



HPC and LINUX: A POWERFUL PARTNERSHIP.